Actividad 6: Taller Práctico de Repaso y Consolidación en C++

Leon Melgarejo

EEST N.º 1 - "Eduardo Ader" Vicente López

Curso: Laboratorio de Programación 5° año 3° - Grupos A-B

Profesor: Mansilla Muñoz York Elías

Fecha: 1 de Octubre de 2026

El presente informe expone los fundamentos y resultados de la Actividad 6, un taller de consolidación analítica diseñado para fijar los tres pilares de bajo nivel en C++: recursividad, arreglos con búsqueda secuencial e intercambio físico por punteros.

Asimismo, se aborda una competencia crítica de ingeniería de software: la implementación de una infraestructura de control de versiones inmune a caídas del servidor mediante la configuración de repositorios remotos redundantes (Dual-Remote) distribuidos en GitHub y GitLab en paralelo. Este taller provee las destrezas lógicas necesarias para procesar flujos de información procedentes de hardware y sensores en tiempo real dentro del Proyecto Integrador Anual (P.I.A.).

Desarrollo

1. Análisis del Código Fuente e Ingeniería de Bajo Nivel

El programa unifica tres retos técnicos operados de forma directa sobre la memoria RAM:

Reto 1 (Recursividad): A través de la función `sumaRecursiva(int n)`, el flujo se invoca a sí mismo dividiendo el problema. La arquitectura reserva memoria en la pila (*Stack*) para cada llamada activa. Se implementa de forma obligatoria un caso base (`if (n <= 0) return 0;`) para asegurar el fin del ciclo autorreferencial.

Reto 2 (Búsqueda Secuencial): Modela un arreglo contiguo en la RAM de 10 enteros (`int vectorDatos[TAM]`), donde las celdas se posicionan de manera consecutiva a una distancia fija de 4 bytes entre sí. El algoritmo realiza un barrido indexado de complejidad lineal $O(n)$. Al hallar el elemento buscado, ejecuta la instrucción `break` para interrumpir el bucle de forma inmediata, optimizando los ciclos de reloj del microprocesador.

Reto 3 (Intercambio con Punteros): La función `intercambiarValores(int* ptrA, int* ptrB)` recibe las ubicaciones físicas de las variables originales empleando el operador de dirección (`&x`, `&y`). Dentro de la función, el operador de desreferenciación (`*`) actúa directamente sobre el valor guardado en las direcciones mutando los datos originales fuera de su ámbito local.

2. Configuración de Infraestructura Dual-Remote

Para mitigar pérdidas accidentales de código ante eventuales caídas del servidor principal (GitHub), se estableció un enlace redundante hacia GitLab mediante PowerShell utilizando los siguientes comandos lógicos:

```
/* ========================================================================

   Materia: Laboratorio de Programación (LPR) — 5°

   Institución: E.E.S.T. N° 20 "Bangtan Sonyeondan" - Vicente López

   Archivo: src/main.cpp

   Actividad 6: Suite de Taller de Repaso e Integración

   ========================================================================
*/



#include <iostream>

using namespace std;



// Prototipos de funciones modulares

int sumaRecursiva(int n);

void intercambiarValores(int* ptrA, int* ptrB);



int main() {

    cout << "=======================================================" <<
endl;

    cout << "  TALLER INTEGRADOR REPASO - ESTUDIANTE: leon melgarejo
" << endl;

    cout << "=======================================================" <<
endl;
```

```
    // --- RETO 1: RECURSIVIDAD ---
    cout << "\n=== RETO 1: SUMA RECURSIVA ===" << endl;
    int num = 5;
    cout << "Suma acumulada de 1 hasta " << num << ": " << sumaRecursiva(num) << endl;

    // --- RETO 2: ARRAYS Y BÚSQUEDA SECUENCIAL ---
    cout << "\n=== RETO 2: BÚSQUEDA EN MEMORIA CONTIGUA ===" << endl;
    const int TAM = 10;
    int vectorDatos[TAM] = {12, 45, 78, 90, 122, 5, 18, 33, 67, 99};
    int numeroBuscar = 18;
    bool hallado = false;

    for (int i = 0; i < TAM; i++) {
        if (vectorDatos[i] == numeroBuscar) {
            cout << "[SISTEMA] Valor " << numeroBuscar << " hallado en el indice [" << i << "]" << endl;
            cout << "          Direccion RAM Hexadecimal: " << &vectorDatos[i] << endl;
            hallado = true;
            break; // Interrupción optimizada del ciclo O(n)
        }
    }
```

```
    if (!hallado) {
        cout << "[SISTEMA] El valor " << numeroBuscar << " no existe en el arreglo." << endl;
    }

    // --- RETO 3: INTERCAMBIO CON PUNTEROS ---
    cout << "\n=== RETO 3: INTERCAMBIO DE CELDAS ===" << endl;
    int x = 100, y = 500;
    cout << "Valores previos -> X: " << x << " | Y: " << y << endl;

    // Invocación pasando direcciones de memoria
    intercambiarValores(&x, &y);
    cout << "Valores actuales -> X: " << x << " | Y: " << y << endl;
    cout << "=====================================================" << endl;

    return 0; // Código 0: Ejecución exitosa
}

// Desarrollo de la función recursiva con caso base garantizado
int sumaRecursiva(int n) {
    if (n <= 0) return 0; // Caso base obligatorio para prevenir Stack Overflow
    return n + sumaRecursiva(n - 1); // Llamada recursiva
}
```

```
// Desarrollo del intercambio físico mediante desreferenciación

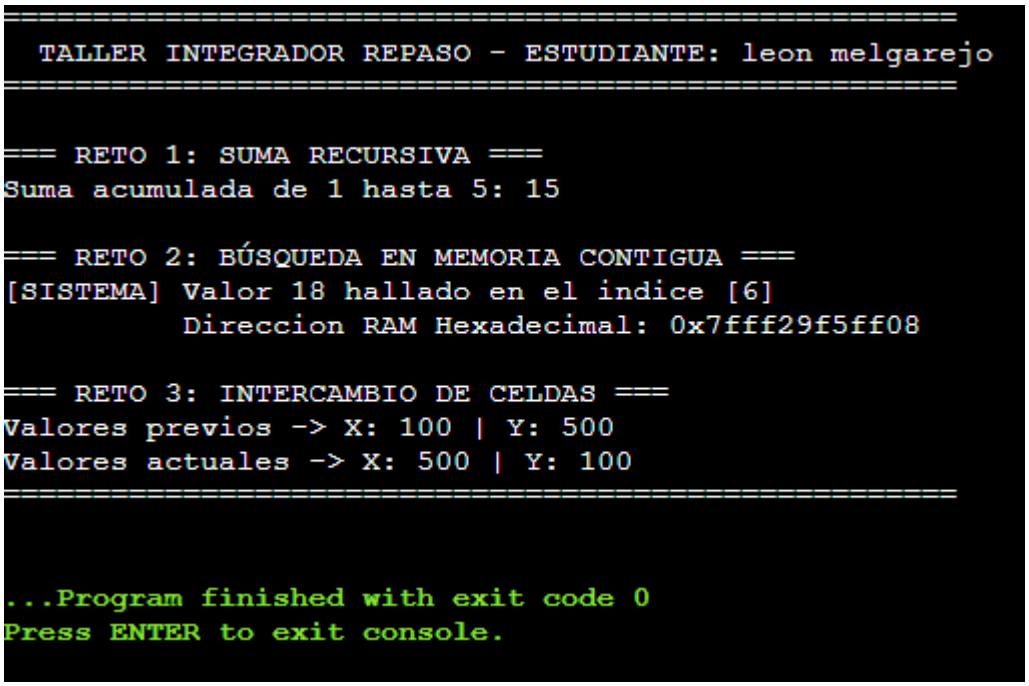
void intercambiarValores(int* ptrA, int* ptrB) {

    int temporal = *ptrA;

    *ptrA = *ptrB;

    *ptrB = temporal;

}
```



A partir de este hito, cada instrucción `git push -u origin main` impacta y actualiza ambas plataformas de forma atómica y simultánea.

Cuestionario Técnico de Control

1. Caso Base: ¿Qué error crítico ocurre en la memoria RAM si una función recursiva carece de un caso base de corte?

Si se omite el caso base, la función genera llamadas autorreferenciales infinitas. Cada invocación empuja un nuevo marco de activación en la pila (*Stack*), almacenando variables locales y direcciones de retorno. Al no liberarse nunca ese espacio, la memoria asignada al programa se agota por completo, desencadenando un fallo crítico por desbordamiento de pila conocido como Stack Overflow, lo que provoca la finalización abrupta del software por parte del sistema operativo.

2. Eficiencia: ¿Por qué la instrucción break optimiza el rendimiento del microprocesador al realizar una búsqueda en arreglos?

En una búsqueda sobre un arreglo de tamaño n, el peor de los casos requiere n comparaciones (complejidad $O(n)$). Si el elemento objetivo se localiza en las primeras posiciones, continuar recorriendo el resto de las celdas vacías consume ciclos de procesamiento inútiles. La instrucción `break` detiene el flujo del bucle en el instante exacto del hallazgo, ahorrando operaciones lógicas innecesarias y devolviendo el control al microprocesador de forma óptima.

3. Direccionamiento: ¿Cómo comprobar mediante el operador & que los elementos de un array ocupan celdas contiguas en la RAM?

Se puede comprobar imprimiendo en la consola las direcciones hexadecimales de celdas sucesivas mediante el operador de dirección (`&vectorDatos[i]`). Al inspeccionar los valores numéricos de las direcciones impresas (por ejemplo, de `&vectorDatos[0]` a `&vectorDatos[1]`), se observa una progresión aritmética exacta de 4 bytes en hexadecimal. Esta diferencia constante corrobora físicamente que el compilador ha reservado un bloque de memoria contiguo en la RAM para el arreglo.

4. Variables Temporales: ¿Qué función cumple la variable temporal en el algoritmo de intercambio de valores con punteros?

La variable temporal actúa como un área de resguardo seguro para los datos. Si se intentara hacer un intercambio directo mediante la asignación `*ptrA = *ptrB;`, el valor almacenado originalmente en la dirección apuntada por `ptrA` se sobrescribiría y se perdería para siempre, impidiendo asignárselo a `ptrB`. Al almacenar primero `*ptrA` en una variable intermedia local (`int temporal = *ptrA;`), se preserva el dato original permitiendo completar el intercambio de manera exitosa.

5. Autoevaluación: ¿Cuál de los 3 retos te resultó más complejo de comprender a bajo nivel y de qué manera lograste resolverlo?

El reto más complejo de asimilar fue el comportamiento de la Recursividad y su interacción con la Pila de Memoria (Stack). Resultaba abstracto visualizar cómo se acumulaban los marcos de ejecución sin resolver la operación matemática de inmediato. Se logró resolver mediante el diseño manual de un diagrama de bloques de la memoria RAM (`traza_memoria.png`), graficando paso a paso cómo crece el Stack con cada llamada y cómo se reduce en cascada al alcanzar el caso base.

Conclusión

La culminación de este taller consolida los conocimientos técnicos sobre el gobierno de la memoria RAM de bajo nivel en C++. Se evidencia que algoritmos eficientes dotados de interrupciones oportunas (`break`) e intercambios directos por dirección (`*`) disminuyen la huella informática en el hardware, un requisito indispensable al programar sistemas embebidos de lectura en tiempo real. Por otra parte, la adopción de una arquitectura de subida Dual-Remote institucionaliza buenas prácticas de DevOps en el equipo de desarrollo, garantizando la alta disponibilidad del código frente a contingencias técnicas de cara a la Expotécnica 2026.

Referencias

EEST N.º 1 "Eduardo Ader". (2026). *Actividad 6: Taller práctico de repaso y consolidación en C++*. Laboratorio de Programación, 5° año 3°.

GeeksforGeeks. (2026). *Linear Search Algorithm*. Algoritmo de Búsqueda Secuencial en Arreglos Contiguos. geeksforgeeks.org

Git Documentation. (2026). *Git Remote — Dual Push Configurations*. Gestión avanzada de repositorios y copias de seguridad redundantes. git-scm.com

Microsoft Corporation. (2026). *Arrays in C++*. Documentación oficial sobre la estructura y direccionamiento físico de arreglos contiguos. microsoft.com

https://github.com/letan-doto07/actividad6-taller-practico-de-repaso-y-consolidacion